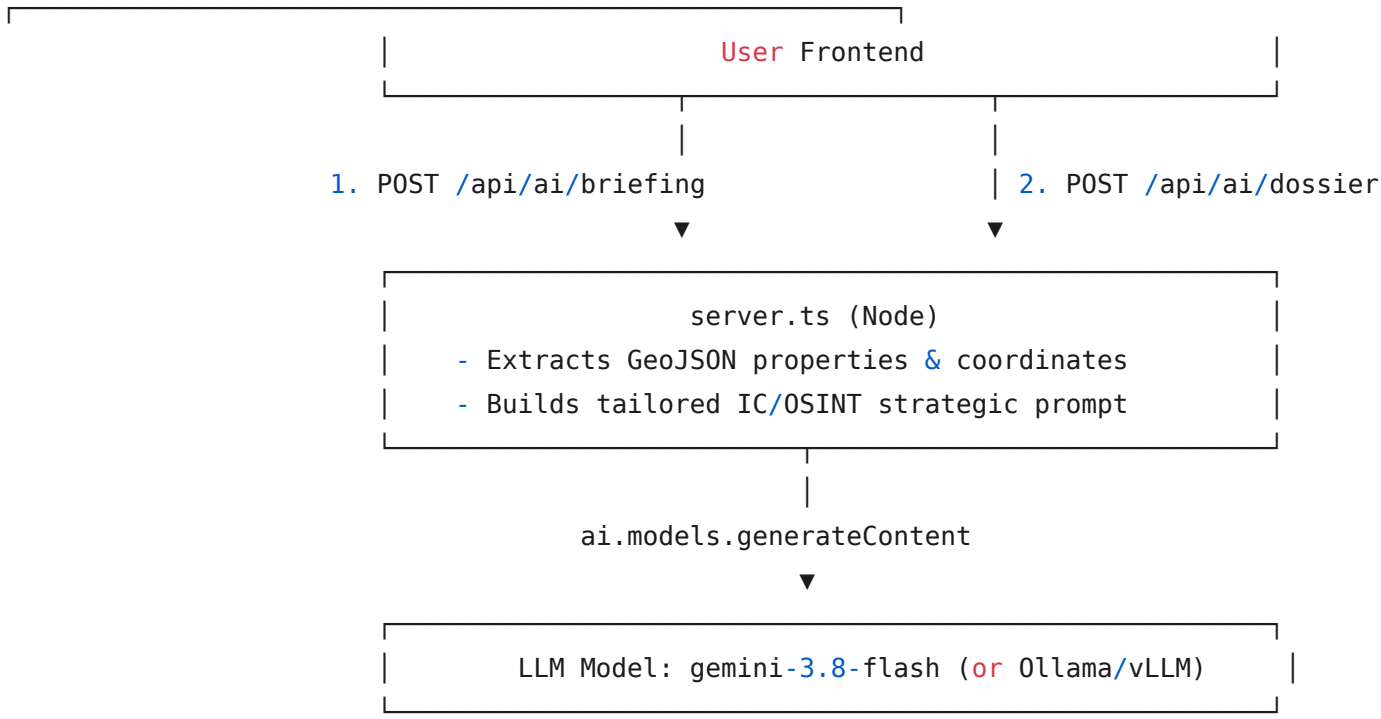
Summary of the Two LLM Contact Points

The backend server (`server.ts`) now provides two dedicated endpoints powered by the `@google/genai` SDK using `gemini-3.8-flash` (with automated resilient fallback to `gemini-flash-latest` and `gemini-3.1-flash-lite`, and a graceful deterministic offline fallback if no API key is set):

code Code



Contact Point 1: Strategic OSINT Briefing (`BriefingGenerator.tsx`)

- **Route:** `POST /api/ai/briefing`
- **Default Model:** `gemini-3.8-flash`
- **Input Context:** Strictly filtered to signals with `stakeholder_relevance: "High"`:

code JSON

```
[
  {
    "id": "leo:2026-09-28:S02",
    "title": "SpaceX Completes Starship Flight 14...",
    "date": "2026-09-28",
    "domain": "Technological",
    "location": "Boca Chica, Texas, USA",
    "coordinates": [-97.1558, 25.9968],
    "horizon": "Now",
    "confidence": "High",
    "source": "https://..."
  }
]
```

- **Tailored Prompt Directives:**
  1. **Operational Executive Synthesis:** Deep macro trend analysis and competitive positioning.

2. **Multi-Domain Convergence Vectors:** Correlating cross-cutting vectors (e.g. Technological deployment velocity intersecting with Political/Regulatory clearances).
3. **Strategic Sector Impacts:** Detailed implications for *Sovereign Telecommunications Gateways*, *Orbital Launch Cadence & Economics*, and *Spectrum Governance & STAs*.
4. **Foresight & Decision Directives:** 30–90 day leading monitoring indicators.

- **Frontend Presentation:**
  - Displays a **Gemini 3.8 Flash** badge when AI synthesis is active.
  - Live generated Markdown is displayed directly in the briefing view and exported via **Copy Briefing** and **Download .MD**.
  - If no API key is detected or the upstream is unavailable, it automatically uses the rule-based convergence engine.

Contact Point 2: Strategic Signal Dossier (`SignalDossier.tsx`)

- **Route:** `POST /api/ai/dossier`
- **Default Model:** `gemini-3.8-flash`
- **Input Context:** The selected signal feature (Title, Domain, Subtheme, Location, Coordinates, ISO-3, Horizon, Confidence, and Verified Source URL).
- **Tailored Prompt Directives (JSON Structured Output):**
  1. `situation_summary`: 2–3 concise, fact-grounded sentences detailing the operational reality.
  2. `strategic_implications`: Penetrating assessment of market dominance, regulatory precedent, or sovereign security implications.
  3. `adversary_dynamics`: How competitors, state incumbents, or geopolitical rivals will react.
  4. `monitoring_indicators`: Array of 3 specific leading indicators or regulatory filings to watch next.
- **Frontend Presentation:**
  - Displays a live **Gemini 3.8 Flash** badge with real-time analysis status indicator.
  - Shows animated pulse skeleton while loading.
  - Gracefully falls back to the deterministic analysis if running offline.

Model Agnosticism & Swapping Models Locally

1. Running with Gemini (Google AI Studio)

In your local Docker or `.env` file, supply your Gemini API key:

code Bash

```
docker run -d -p 3000:3000 \
  -e GEMINI_API_KEY="your-gemini-api-key" \
  -v "$(pwd)/data:/app/data" \
  osint-signals-map
```

2. Running Model-Agnostic with Local LLMs (Ollama / vLLM / LocalAI)

Because both prompts use standard JSON context and receive Markdown or structured JSON, you can swap Gemini for any local model by redirecting the two calls in `server.ts`:

- **Equivalent Local Model Capacities:**

- **Ollama / Local:** `llama3.3:70b`, `qwen2.5:32b`, or `mistral-small:24b` (excellent for multi-signal intelligence synthesis).
- **OpenAI:** `gpt-4o-mini` or `gpt-4o`.
- **Anthropic:** `claude-3-5-sonnet` or `claude-3-5-haiku`.

To point `server.ts` to Ollama locally instead of Gemini, replace the `ai.models.generateContent` call in `server.ts`:

code Ts

```ts
const res = await fetch("http://localhost:11434/v1/chat/completions", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "llama3.3:70b",
    messages: [{ role: "user", content: prompt }]
  })
});
```

Complete Docker Run Command with Local GeoJSON & LLM Key

code Bash

```bash
# Place your GeoJSON file in ./data/signals.geojson
mkdir -p data
cp /path/to/my-signals.geojson data/signals.geojson

# Start full-stack container with your Gemini Key:
docker compose up --build -d
```